

Reading from Analog and Encoder Sensors. Outputting to PWM (for NanoRP2040)

Part 1: Reading an Analog Signal

Objective:

- Read analog input position of the potentiometer
- Control the brightness of a LED using pulse width modulation (PWM)
- Understand how to read a quadrature encoder works

Background Information:

Analog to Digital Converter (ADC):

Arduino's onboard *analog to digital converter (ADC)* converts an analog signal to a digital value. The following equation relates the input voltage to the converted digital value:

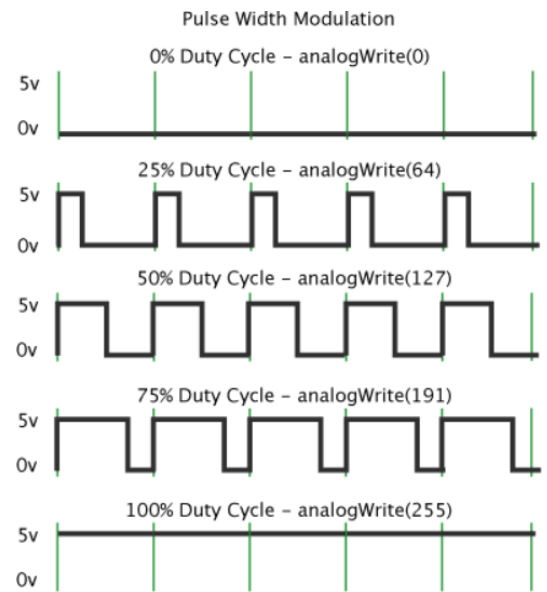
$$\frac{A/D_{result}}{ADC_{resolution}} = \frac{V_{in}}{V_{ref}}$$

- V_{in} is the input voltage – typically from a sensor or in this case a potentiometer
- V_{ref} is the reference voltage (usually 5v or 3.3v)
- $ADC_{resolution}$ is the resolution of the ADC conversion, in this case it is 12 bits (4096)

Question: If the value of the ADC on A0 of the Arduino is 512, what is the input voltage on A0 (assume the Arduino operates at 3.3v)?

Pulse Width Modulation (PWM)

If the microcontroller does not have Digital to Analog Converter (DAC) then it can only output a binary high or low voltage (3.3v or 0v). *Pulse width modulation (PWM)* is used to create an *average* voltage by varying the duty cycle, or the amount of time the signal is on/off. A 100% duty cycle corresponds to fully powering the LED with 5v and a 0% duty cycle means the LED will receive 0v. A 50% duty cycle means *on average* the voltage seen by the LED is 2.5v. The value assigned to the PWM block can be from 0 to 255: 100% duty cycle corresponding to 255 and 0% duty cycle corresponding to 0.



PWM duty cycle
(<https://www.arduino.cc/en/Tutorial/PWM>)

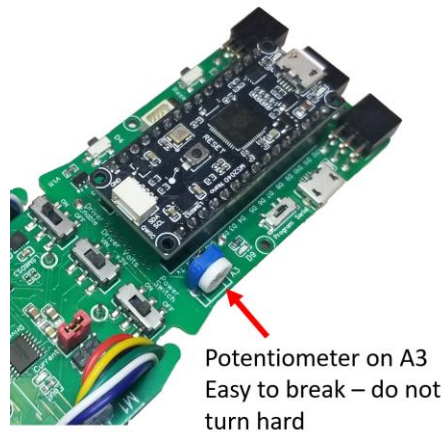
Simulink Model

It is easiest to start with a Simulink model that is already set up for hardware implementation. The demo files for your hardware, or the file you set up in Lab 1 should be used for this purpose. You can always run the demo file if you think you might have a hardware problem.

Delete everything in this diagram except the blocks you want to use. (these blocks have different names depending on your hardware system).

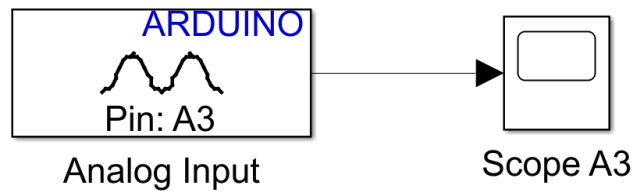
Analog Input

The potentiometer on the MinSegNano is the white circle located on the side of the Shield. Rotating it changes the resistance/voltage read by pin A3:



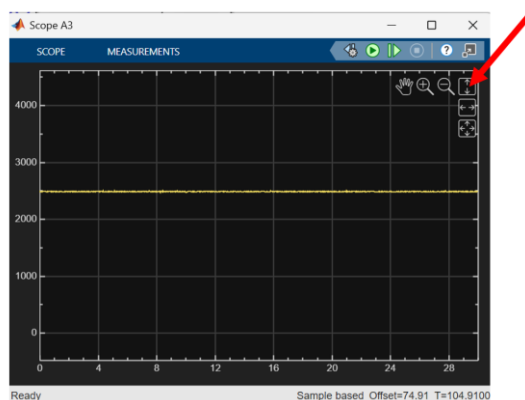
This potentiometer is fragile – do not try to turn it hard past its limit – look at the scope to view if it is approaching the limit: 0-4096

Build and run the following Simulink diagram with the following settings.

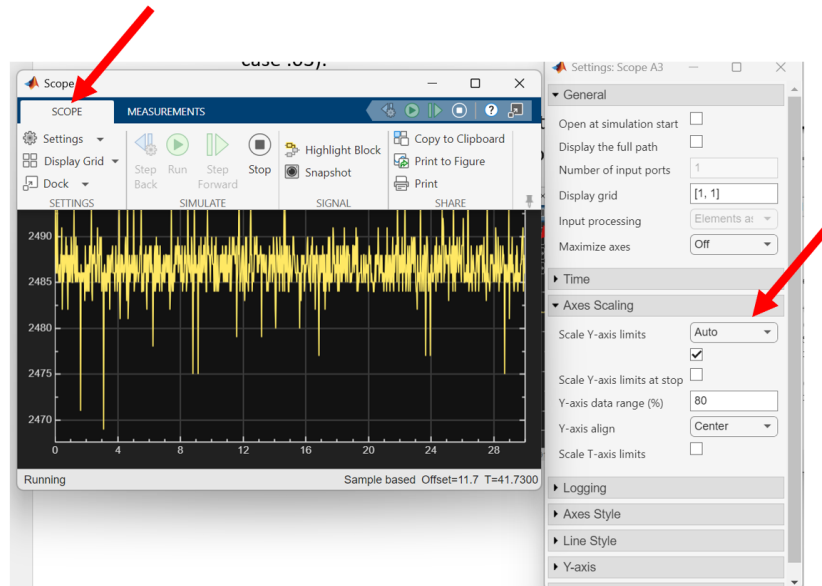


- **Configuration Parameters:** The solver “Fixed-step size” should be .03 seconds.
- **Analog Input:** Set the Pin number to 3 and the sampling time to -1. When the sample time is -1 the block will inherit the sample time used in the simulation settings (in this case .03).

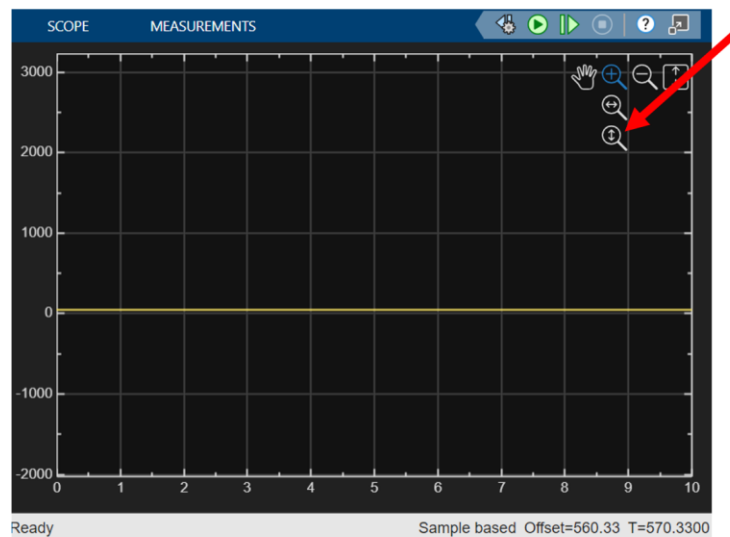
Observe the scopes while changing the position of the potentiometer wiper. Hover in the upper right hand side of the scope to see the option for autoscale:



Automatic autoscale can be selected instead of manually clicking on the button. Click on the Scope text at the upper left of the scope window:

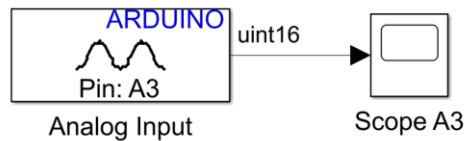
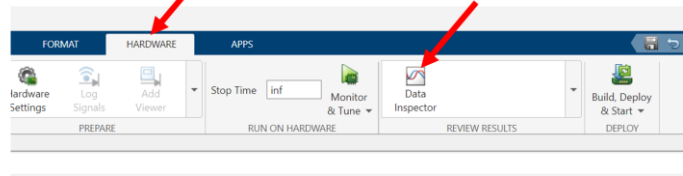


Note: Autoscale in 2025a will not *zoom in* correctly, it will only zoom out. To zoom in manually hover in the upper right hand corner and select the magnifying glass with the up and down arrow:

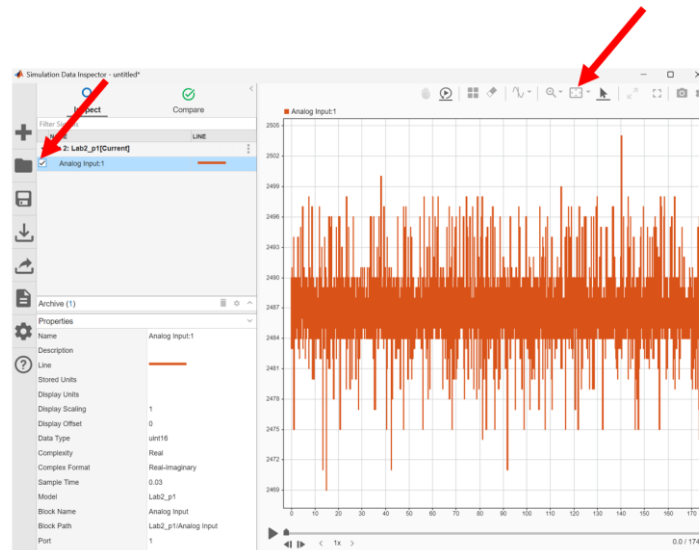


Then drag over the range of the graph you want to zoom in on.

To view data with an autoscale function in 2025a use the Data Inspector in Simulink: Click the Hardware Tab at the top then select the Data Inspector.



Select the signal you want to view, then you can use autoscale buttons in Data Inspector which will automatically zoom in on the signal when you click it:

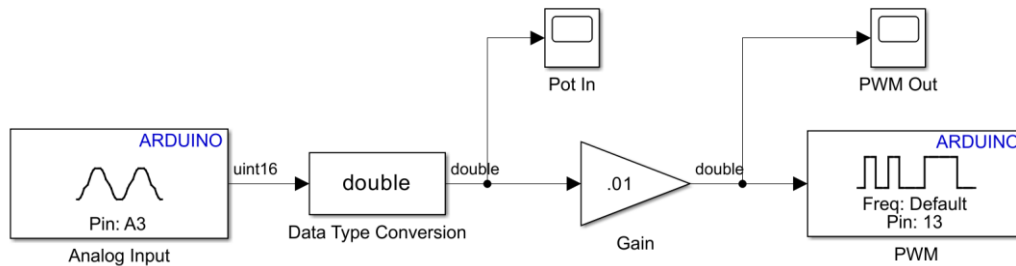


Make sure you are familiar with looking at data with both the scope and the Data Inspector.

Questions: Provide a screenshot of the Data Inspector plot of the analog input as you change the potentiometer position. What is the max and minimum values received?

PWM Output

Build and run the following Simulink diagram with the following settings.



The “Pot in” and “PWM out” blocks both use the ‘Scope’ block.

- **Configuration Parameters:** The “Fixed-step size” should be 0.03 seconds.
- **Analog Input:** Set the Pin number to 3 and the sampling time to -1. When the sample time is -1 the block will inherit the sample time used in the simulation settings - in this case 0.03.
- **PWM:** Set the Pin to 13 (the onboard LED).
- **Gain:** Set the gain to the correct value so the analog input’s full range will utilize the full PWM output
 - Hint: what is the maximum digital value for the PWM block and the maximum value for the ADC?
- **Data Type Conversion:** Go to Simulink-> Signal Attributes. Double click this block and set output data type to double. This ensures floating point math is performed to avoid truncation and division errors associated with fixed point numbers.

Question: What is the ideal correct value for the gain such that the full range of the potentiometer will utilize the full range of the PWM (you can leave this as a fraction)?

Part 2: Reading Encoders

Objective:

For Part 2, an encoder will be used to determine the position of the output motor shaft. The brightness of the LED will be controlled based on the position.

Background Information NXT Motor Encoder:

A Lego NXT motor has an encoder wheel inside that measures the rotation of the motor shaft. An encoder is a device that can be used to determine position. Most encoders use optical sensors to output a pulse train which can be decoded to determine position and direction. In this lab, the encoder will be used to track angular position.



Figure 1: Lego NXT Motor

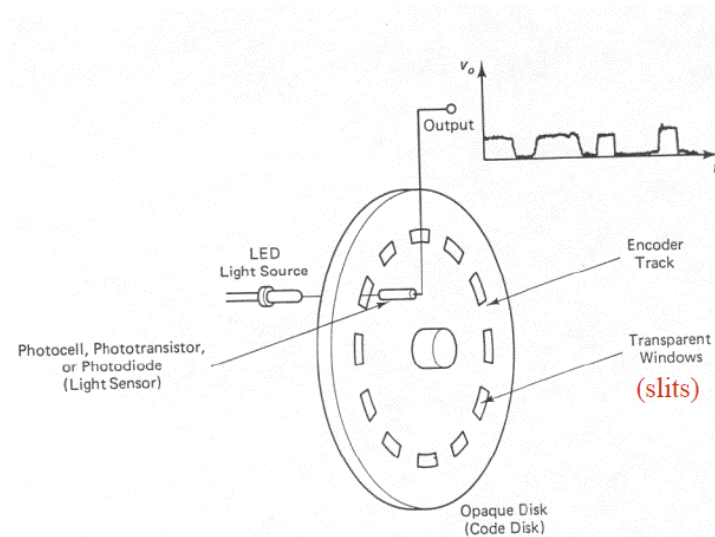


Figure 2: Example of Optical Encoder signals

- The NXT encoder has a raw resolution of 12 counts. The figure on the left below shows the actual encoder wheel in the NXT motor. It has 12 holes. The optical sensor will output a high or low (1 or 0) depending on the position of this encoder. Each revolution will produce 12 pulses.

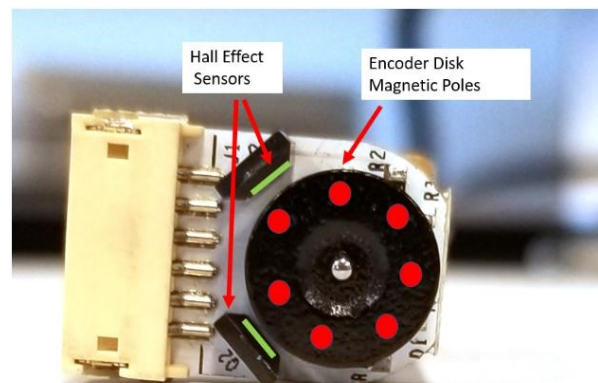
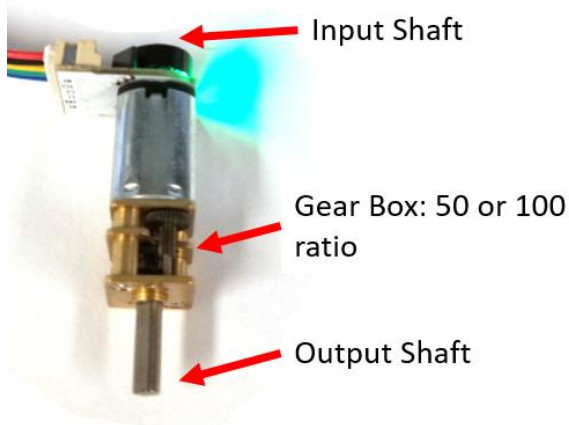


- If the output motor shaft moves 1 revolution the encoder wheel turns 15 revolutions – this means for **one revolution of the motor output shaft there will be $15 \times 12 = 180$ pulses from the encoder.**
- The encoder device provides two channels A & B. These channels are the encoder pulses with one channel shifted by 90 degrees. By monitoring these two channels both the position and direction of the motor shaft can be determined. **If quadrature decoding is used the resolution can be increased by 4 times.**
- The Encoder block included with this lab performs quadrature decoding so the total resolution is:

$$\text{Angular Resolution} = 15 * 12 * 4 = 720 \text{ pulses per revolution of output shaft}$$

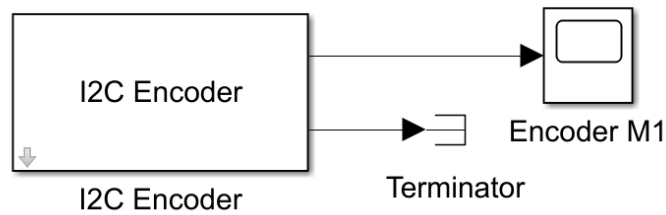
Background Information Micro Gear Motor:

The encoder for the micro gear motor uses an encoder disk that has magnets placed in the round disk. There are 7 for this encoder disk. The two hall effect sensors measure the poles as they pass the sensor they provide an On/Off signal to the Arduino. For every revolution of the **input** shaft the **quadrature decoded output** will be $4 \times 7 = 28$. For every revolution of the **output** shaft the **quadrature decoded output** will be $4 \times 7 \times Gr$, where Gr is the gear ratio of the gearbox (5, 30, 50 or 100 etc. depending on your motor's gearbox)



Simulink Model

Build and run the following Simulink diagram. The encoder block can be obtained from the Demo file for your hardware located in the Examples folder of RASPLib.



Turn the motor output shaft. Observe that the scope captures the change in the motor position.

Questions: Rotate the output motor shaft 1 revolution (approximately). How much does recorded position change? What is the gear ratio for your motor? Provide a screenshot of your graph that shows how the values change with 1 rotation.